

PW EARNERS
DSA in C++

DPP — 6 (SOLUTIONS)
Topic: Arrays and Vectors in C++

Q1. State **True** or **False** for each statement below regarding Arrays and Vectors.

- (i) The size of a static array in C++ can be dynamically changed during run-time after its declaration. **False**
- (ii) Attempting to access `arr[-1]` in C++ will always trigger a direct compilation error. **False**
- (iii) The expression `sizeof(arr)/sizeof(arr[0])` accurately calculates the number of elements in a static array. **True**
- (iv) When an array is passed to a function using `void solve(int arr[])`, it is fundamentally passed by reference (as a pointer). **True**
- (v) The statement `vector<int> v(5, 10);` initializes a vector of size 5, where every element holds the value 10. **True**
- (vi) The `capacity()` of a vector in C++ is always strictly equal to its `size()`. **False**
- (vii) Elements of a standard C++ array are allocated in contiguous memory locations. **True**

Multiple Choice Questions (Theoretical Understanding)

Q2. Correct Option: **A**

Explanation: `vector<int> v(N, 0);` correctly initializes a dynamic array of size N with all elements initialized to 0.

.....

Q3. Correct Option: **C**

Explanation: Vectors manage dynamic memory automatically. When capacity becomes full, they allocate a new, larger memory block (often double the previous capacity), copy existing elements over, and free the old memory.

.....

Q4. Correct Option: **B**

Explanation: The member function `pop_back()` is standard in C++ STL for removing the last element of a vector.

.....

Q5. Correct Option: **A**

Explanation: While memory reallocation occasionally takes $O(N)$ time, the amortized (aver-

age over many insertions) time complexity of `push_back()` is $O(1)$.

.....

Q6. Correct Option: B

Explanation: In C++, an array identifier used without an index automatically decays into a pointer pointing to the memory address of its first element (`&arr[0]`).

.....

Q7. Correct Option: B

Explanation: Passing by reference (`&`) avoids copying the vector elements, allowing the function to inspect and modify the original vector directly in memory.

Q8. Output: 20 0

Explanation: When an array is partially initialized with an initializer list, C++ aggregate initialization guarantees that all uninitialized remaining elements are automatically set to 0 (not garbage values). Thus, `arr[1]` is 20 and `arr[3]` is 0.

.....

Q9. Output: 2 4

Explanation: 3 elements are pushed into the vector (capacity grows dynamically: $1 \rightarrow 2 \rightarrow 4$). Calling `pop_back()` removes the last element, reducing the size to 2, while the allocated capacity remains 4.

.....

Q10. Output: 1

Explanation: The vector is passed by *value*. A temporary copy is created and modified inside `updateVector()`, leaving the original vector in `main` unchanged.

.....

Q11. Output: 100

Explanation: Plain arrays decay to pointers when passed into functions, passing them fundamentally by reference. Thus, modifying `arr[0]` directly updates the original array in `main`.

.....

Q12. Output: Garbage Value / Undefined Behavior

Explanation: The array is declared with size 3 (valid indices are 0, 1, and 2). Attempting to access `arr[4]` accesses out-of-bounds memory, leading to undefined behavior or outputting a garbage value.

.....

Q13. Output: 5 10

Explanation: `vector<int> v(3, 5)` initializes 3 elements with value 5. `push_back(10)` appends 10 at index 3. Index 2 holds 5, and index 3 holds 10.

.....

Q14. Output: 20 30

Explanation: `*(arr + 1)` uses pointer arithmetic to access the element at index 1 (20). `arr[2]` accesses the element at index 2 (30).

.....

Q15. Output: 5 0

Explanation: `sizeof(arr)` is $5 \times 4 = 20$ bytes, and `sizeof(arr[0])` is 4 bytes, yielding a size of 5. Because the array is partially initialized (`{1, 2}`), remaining elements are implicitly set to 0, making `arr[4]` equal to 0.

Q16. Output: 4

Explanation: *Understanding the Range-Based For Loop (For-Each Loop):* A range-based for loop syntax `for(type variable : container)` iterates sequentially through each element in a collection (like an array or vector). By writing `for(int &x : v)`, `x` is declared as a reference (&) to each element in vector `v`. Modifying `x` (`x = x * 2`) directly doubles the value inside the vector. Element at index 1 changes from 2 to 4.

Q17. Output: 2 6 7

Explanation: 1. `*ptr++`: Post-increment operator evaluates `*ptr` first (2) and then advances `ptr` to point to index 1 (value 4). 2. `++*ptr`: Pre-increment operator advances `ptr` first to point to index 2 (value 6) and evaluates to 6. 3. `++*ptr`: Pre-increment on dereferenced value increments the value at index 2 in-place from 6 to 7.

Q18. Output: Undefined Behavior (Dangling Reference)

Explanation: `ref` stores a reference to `v[0]`. Triggering multiple `push_back()` calls forces the vector to reallocate its underlying memory buffer. `ref` is left pointing to invalidated memory (a dangling reference), causing undefined behavior.

Q19. Third Maximum

```

1  #include <iostream>
2  #include <vector>
3  #include <climits>
4  using namespace std;
5
6  void findThirdMax(vector<long long>& arr, int n) {
7      long long m1 = LLONG_MIN, m2 = LLONG_MIN, m3 = LLONG_MIN;
8
9      // Traversing array to find 1st, 2nd, and 3rd maximum values by
      // order
10     for (int i = 0; i < n; i++) {
11         long long val = arr[i];
12         if (val >= m1) {
13             m3 = m2;
14             m2 = m1;
15             m1 = val;
16         } else if (val >= m2) {
17             m3 = m2;
18             m2 = val;
19         } else if (val > m3) {
20             m3 = val;
21         }
    }

```

```

22     }
23     cout << m3 << endl;
24 }
25
26 int main() {
27     int n;
28     cin >> n;
29     vector<long long> arr(n);
30     for (int i = 0; i < n; i++) {
31         cin >> arr[i];
32     }
33     findThirdMax(arr, n);
34 }

```

Time Complexity: $O(N)$ | **Space Complexity:** $O(1)$

Note on Problem Statement: There are two ways to interpret "third maximum": finding the *unique* third maximum (ignoring duplicates) or finding the third largest element *by positional order* (allowing duplicates). Since the question does not specify unique elements, we calculate the third maximum element by order.

Explanation: We maintain three variables (m1, m2, m3) to track top elements. As we traverse the array once, values shift down whenever a larger number is encountered.

Q20. Complex Array Rotation

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void reverseArray(vector<int>& arr, int start, int end) {
6      while (start < end) {
7          int temp = arr[start];
8          arr[start] = arr[end];
9          arr[end] = temp;
10         start++;
11         end--;
12     }
13 }
14
15 void optimalRotation(vector<int>& arr, int n, long long X, long long
    Y) {
16     // Net effective rotation amount
17     long long netRight = (X - Y) % n;
18     if (netRight < 0) netRight += n;
19
20     // Reversal Algorithm for right rotation
21     reverseArray(arr, 0, n - 1);
22     reverseArray(arr, 0, netRight - 1);
23     reverseArray(arr, netRight, n - 1);
24
25     for (int i = 0; i < n; i++) {
26         cout << arr[i] << " ";
27     }
28     cout << endl;

```

```

29 }
30
31 int main() {
32     int n;
33     cin >> n;
34     vector<int> arr(n);
35     for (int i = 0; i < n; i++) {
36         cin >> arr[i];
37     }
38     long long x, y;
39     cin >> x >> y;
40     optimalRotation(arr, n, x, y);
41 }

```

Time Complexity: $O(N)$ | **Space Complexity:** $O(1)$

Intuition: Right rotating by X and left rotating by Y simplifies mathematically to a net right rotation of $(X - Y) \pmod N$. We use the 3-step Reversal Algorithm to achieve in-place $O(N)$ rotation.

Q21. Strictly Sorted Non-Increasing

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  bool isNonIncreasing(vector<int>& arr, int n) {
6      for (int i = 1; i < n; i++) {
7          if (arr[i] > arr[i - 1]) return false;
8      }
9      return true;
10 }
11
12 int main() {
13     int n;
14     cin >> n;
15     vector<int> arr(n);
16     for (int i = 0; i < n; i++) {
17         cin >> arr[i];
18     }
19
20     if (isNonIncreasing(arr, n) == true) {
21         cout << "true" << endl;
22     } else {
23         cout << "false" << endl;
24     }
25 }

```

Time Complexity: $O(N)$ | **Space Complexity:** $O(1)$

Explanation: We iterate from index 1 to $N - 1$ and check if any current element is strictly larger than its predecessor. If found, it returns false.

Q22. Count Elements Strictly Less

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int countLessThanT(vector<int>& arr, int n, int target) {
6     int count = 0;
7     for (int i = 0; i < n; i++) {
8         if (arr[i] < target) {
9             count++;
10        }
11    }
12    return count;
13 }
14
15 int main() {
16     int n, t;
17     cin >> n >> t;
18     vector<int> arr(n);
19     for (int i = 0; i < n; i++) {
20         cin >> arr[i];
21     }
22     cout << countLessThanT(arr, n, t) << endl;
23 }
```

Time Complexity: $O(N)$ | **Space Complexity:** $O(1)$

Explanation: A standard linear scan that increments a counter whenever an element is strictly smaller than target value T .

.....

Q23. First Repeated Element

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int firstRepeated(vector<int>& arr, int n) {
6     // Frequency array (Max constraint arr[i] <= 10^5)
7     vector<int> freq(100001, 0);
8
9     for (int i = 0; i < n; i++) {
10         freq[arr[i]]++;
11     }
12
13     for (int i = 0; i < n; i++) {
14         if (freq[arr[i]] > 1) {
15             return arr[i];
16         }
17     }
18
19     return -1;
20 }
21
22 int main() {
```

```

23     int n;
24     cin >> n;
25     vector<int> arr(n);
26     for (int i = 0; i < n; i++) {
27         cin >> arr[i];
28     }
29     cout << firstRepeated(arr, n) << endl;
30 }

```

Time Complexity: $O(N)$ | **Space Complexity:** $O(M)$ where $M = 10^5$

Explanation: We populate frequency counts using an array. In the second pass from left to right, the first element with frequency > 1 is returned.

Q24. Move Zeroes to End

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void moveZeroes(vector<int>& arr, int n) {
6      int nonZeroIndex = 0;
7
8      // Shift non-zero elements to front
9      for (int i = 0; i < n; i++) {
10         if (arr[i] != 0) {
11             arr[nonZeroIndex] = arr[i];
12             nonZeroIndex++;
13         }
14     }
15
16     // Fill trailing positions with zero
17     for (int i = nonZeroIndex; i < n; i++) {
18         arr[i] = 0;
19     }
20 }
21
22 int main() {
23     int n;
24     cin >> n;
25     vector<int> arr(n);
26     for (int i = 0; i < n; i++) {
27         cin >> arr[i];
28     }
29
30     moveZeroes(arr, n);
31     for (int i = 0; i < n; i++) {
32         cout << arr[i] << " ";
33     }
34     cout << endl;
35 }

```

Time Complexity: $O(N)$ | **Space Complexity:** $O(1)$

Intuition: Using a two-pointer approach, `nonZeroIndex` keeps track of where the next non-

zero element should reside, preserving relative ordering cleanly.

.....

Q25. Check Palindrome Array

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 bool isPalindrome(vector<int>& arr, int n) {
6     int left = 0, right = n - 1;
7     while (left < right) {
8         if (arr[left] != arr[right]) return false;
9         left++;
10        right--;
11    }
12    return true;
13 }
14
15 int main() {
16     int n;
17     cin >> n;
18     vector<int> arr(n);
19     for (int i = 0; i < n; i++) {
20         cin >> arr[i];
21     }
22
23     if (isPalindrome(arr, n)) cout << "Palindrome" << endl;
24     else cout << "Not Palindrome" << endl;
25 }
```

Time Complexity: $O(N)$ | **Space Complexity:** $O(1)$

Explanation: Compares mirror elements starting from both ends towards the center.

.....

Q26. Count Even and Odd Numbers

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 void countEvenOdd(vector<int>& arr, int n) {
6     int evens = 0, odds = 0;
7     for (int i = 0; i < n; i++) {
8         if (arr[i] % 2 == 0) evens++;
9         else odds++;
10    }
11    cout << "Even: " << evens << ", Odd: " << odds << endl;
12 }
13
14 int main() {
15     int n;
16     cin >> n;
17     vector<int> arr(n);
```

```

18     for (int i = 0; i < n; i++) {
19         cin >> arr[i];
20     }
21     countEvenOdd(arr, n);
22 }

```

Time Complexity: $O(N)$ | **Space Complexity:** $O(1)$

Explanation: Traverses using index loops and checks parity using modulo operator (% 2).

.....

Q27. Difference Array

```

1  #include <iostream>
2  #include <vector>
3  #include <cmath>
4  using namespace std;
5
6  vector<int> getDifferenceArray(vector<int>& arr, int n) {
7      vector<int> diff(n);
8      for (int i = 0; i < n - 1; i++) {
9          diff[i] = abs(arr[i] - arr[i + 1]);
10     }
11     diff[n - 1] = abs(arr[n - 1] - arr[0]);
12     return diff;
13 }
14
15 int main() {
16     int n;
17     cin >> n;
18     vector<int> arr(n);
19     for (int i = 0; i < n; i++) {
20         cin >> arr[i];
21     }
22
23     vector<int> res = getDifferenceArray(arr, n);
24     for (int i = 0; i < n; i++) {
25         cout << res[i] << " ";
26     }
27     cout << endl;
28 }

```

Time Complexity: $O(N)$ | **Space Complexity:** $O(N)$

Explanation: Creates a new array storing absolute differences between adjacent elements, explicitly handling circular calculation for the last element.

.....

Q28. Absolute Difference of Sums

```

1  #include <iostream>
2  #include <vector>
3  #include <cmath>
4  using namespace std;
5
6  int getSumDifference(vector<int>& arr, int n) {

```

```

7     long long evenSum = 0, oddSum = 0;
8     for (int i = 0; i < n; i++) {
9         if (i % 2 == 0) evenSum += arr[i];
10        else oddSum += arr[i];
11    }
12    return abs(evenSum - oddSum);
13 }
14
15 int main() {
16     int n;
17     cin >> n;
18     vector<int> arr(n);
19     for (int i = 0; i < n; i++) {
20         cin >> arr[i];
21     }
22     cout << getSumDifference(arr, n) << endl;
23 }

```

Time Complexity: $O(N)$ | **Space Complexity:** $O(1)$

Explanation: Accumulates sums separately depending on whether index i is even or odd, then returns absolute difference.

Q29. Minimum Absolute Difference

```

1  #include <iostream>
2  #include <vector>
3  #include <cmath>
4  #include <climits>
5  using namespace std;
6
7  int minAdjacentDiff(vector<int>& arr, int n) {
8      int minDiff = INT_MAX;
9      for (int i = 0; i < n - 1; i++) {
10         int diff = abs(arr[i] - arr[i + 1]);
11         if (diff < minDiff) {
12             minDiff = diff;
13         }
14     }
15     return minDiff;
16 }
17
18 int main() {
19     int n;
20     cin >> n;
21     vector<int> arr(n);
22     for (int i = 0; i < n; i++) {
23         cin >> arr[i];
24     }
25     cout << minAdjacentDiff(arr, n) << endl;
26 }

```

Time Complexity: $O(N)$ | **Space Complexity:** $O(1)$

Explanation: Iterates over consecutive pairs, keeping track of the minimum difference found

so far.

.....

Q30. Max Consecutive Ones

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 int findMaxConsecutiveOnes(vector<int>& arr, int n) {
7     int maxCount = 0;
8     int currentCount = 0;
9     for (int i = 0; i < n; i++) {
10         if (arr[i] == 1) {
11             currentCount++;
12             maxCount = max(maxCount, currentCount);
13         } else {
14             currentCount = 0;
15         }
16     }
17     return maxCount;
18 }
19
20 int main() {
21     int n;
22     cin >> n;
23     vector<int> arr(n);
24     for (int i = 0; i < n; i++) {
25         cin >> arr[i];
26     }
27     cout << findMaxConsecutiveOnes(arr, n) << endl;
28 }
```

Time Complexity: $O(N)$ | **Space Complexity:** $O(1)$

Explanation: Counts contiguous 1s linearly and resets counter to 0 whenever a 0 is encountered.

All the best! Keep Coding ♥